**MFRAL TECH Innovación y Soluciones Inteligentes**
**“Conectamos ideas con tecnología”**

**Francisco Steven Tabares Ussa - Developer**
**Edward Rubio Rodríguez - DevOps + apoyo general**
**Leonardo Martínez Franco - Product Owner**
**Maria Fernanda Medina Ramirez - QA Engineer + Documentación**

**UNIVERSIDAD AUTONOMA DE OCCIDENTE**
**INGENIERÍA INFORMATICA**
**CALI**
**2026**

Universidad Autónoma de Occidente

MEFRAL TECH
Innovación y Soluciones Inteligentes

## Contenido


1. Introducción ..... 4
2. Arquitectura del sistema ..... 4
   2.1 Descripción General ..... 4
   2.2 Componentes del Sistema ..... 5
   2.3 Tecnologías Utilizadas ..... 6
   2.4 Estructura del sistema ..... 6
   2.5 Flujo de comunicación del sistema ..... 7
3. Implementación del MVP ..... 8
   3.1 Datos de prueba y configuración inicial ..... 8
   3.2 Flujo técnico implementado ..... 8
   3.3 Estado de implementación por requerimiento ..... 10
   3.4 Integración del sistema ..... 10
4. Ejecución de sprints ..... 10
   Sprint 1 – Gestión de usuarios ..... 11
   Sprint 2 – Gestión de productos ..... 11
   Sprint 3 – Catálogo y proceso de compra ..... 12
   4.1 Consolidación de los sprints ..... 13
5. Pruebas funcionales ..... 13
   5.1 Estrategia de pruebas ..... 13
   5.2 Ejecución de pruebas ..... 14
   5.3 Evolución de las pruebas ..... 15
   5.4 Principales hallazgos ..... 15
   5.5 Conclusión de pruebas ..... 17
6. Despliegue del sistema ..... 17
   6.1 Entorno de ejecución ..... 17
   6.2 Configuración del backend ..... 17
   6.3 Configuración del frontend ..... 18
   6.4 Integración con Firebase ..... 18
   6.5 Exposición del sistema en entorno de prueba ..... 19

**6.6 Flujo de despliegue** ..... 19
**6.7 Consideraciones técnicas** ..... 19
**6.8 Estado del despliegue** ..... 19
**7. Gestión del proyecto** ..... 20
**7.1 Organización del tablero** ..... 20
**7.2 Trazabilidad de requerimientos** ..... 20
**7.3 Evidencia de gestión** ..... 20
**7.4 Consideraciones** ..... 24
**8. Roles y responsabilidades del equipo** ..... 24
**8.1 Asignación de roles** ..... 24
**8.2 Consideraciones del equipo** ..... 24
**9. Conclusión** ..... 25
**10. Repositorio** ..... 25
**10.1 Entorno de ejecución** ..... 26
**10.2 Consideraciones** ..... 26
**11. Bibliografía** ..... 26

## Tabla de Ilustraciones

*Ilustración 1- Tablero de gestión de tareas – Sprint 1 (GitHub Projects)* ..... *21*
*Ilustración 2 - Tablero de gestión de tareas – Sprint 1 Avances* ..... *21*
*Ilustración 3 - Estado inicial del tablero de GitHub Projects con avance del Sprint 1 y tareas en desarrollo del Sprint 2* ..... *22*
*Ilustración 4 - Tablero de gestión del proyecto en GitHub Projects con evidencia de ejecución de los Sprints 1, 2 y 3* ..... *22*
*Ilustración 5 - Estado actual del tablero de GitHub Projects con la ejecución de los sprints, evidenciando requerimientos implementados, en progreso y pendientes* ..... *23*
*Ilustración 6 - Estado final del tablero de GitHub Projects evidenciando la finalización de las tareas del Sprint 1 y la ejecución completa de los requerimientos funcionales* ..... *23*

## 1. Introducción

En el marco del desarrollo del proyecto Café Origen, el presente documento corresponde a la evolución técnica del Producto Mínimo Viable (MVP), evidenciando la transición desde una implementación inicial parcialmente funcional hacia una solución completa, estable y validada, durante la ejecución del Avance 2, se identificaron diversas limitaciones en la versión inicial del sistema, particularmente en aspectos relacionados con la arquitectura, la gestión de datos, la estabilidad de los módulos y la completitud del flujo de negocio, estas condiciones, sumadas a la reasignación del rol de desarrollo dentro del equipo, llevaron a la decisión de realizar una reimplementación técnica del sistema desde cero.

Como resultado de este proceso, se desarrolló una nueva versión del aplicativo denominada OrigenX (Conexión Cafetera), la cual introduce mejoras significativas tanto a nivel estructural como funcional, esta nueva versión incorpora una arquitectura más robusta, basada en servicios desacoplados, el uso de tecnologías modernas como Firebase para la gestión de datos y autenticación, y una organización del backend en capas (routes, services y repositories), permitiendo una mayor escalabilidad, mantenibilidad y seguridad, a diferencia de la versión anterior, centrada en la validación inicial del MVP, esta nueva iteración presenta un sistema completamente funcional, que cubre el flujo end-to-end del negocio, incluyendo autenticación de usuarios, gestión de productos, visualización de catálogo, carrito de compras, generación de pedidos y procesamiento de pagos simulados.

En este contexto, el presente documento no solo evidencia el estado actual del sistema, sino también el proceso de evolución técnica, mejora continua y consolidación de la solución, alineado con un enfoque de desarrollo ágil y centrado en la calidad del software.

## 2. Arquitectura del sistema

### 2.1 Descripción General

El sistema OrigenX (Conexión Cafetera) se desarrolla bajo una arquitectura cliente-servidor desacoplada, orientada a servicios, en la cual se separan claramente las responsabilidades entre la capa de presentación, la lógica de negocio y la gestión de datos.

El sistema se compone de los siguientes elementos principales:

- **Frontend (Cliente):** Aplicación web SPA desarrollada en React, encargada de la interacción con el usuario y el consumo de servicios backend.
- **Backend (API):** Servicio REST desarrollado con FastAPI, responsable de la lógica de negocio, validaciones y orquestación de operaciones.
- **Servicios externos (Firebase):** Plataforma utilizada para la autenticación, persistencia de datos y almacenamiento de archivos.

Esta arquitectura permite una comunicación fluida entre componentes mediante protocolos HTTP, facilitando la escalabilidad, mantenibilidad y modularidad del sistema.

### 2.2 Componentes del Sistema

El sistema se organiza en módulos funcionales que responden a las necesidades del negocio

**Módulo de Autenticación**

- Registro de usuarios mediante Firebase Authentication
- Inicio de sesión con validación de credenciales
- Gestión de roles mediante custom claims (Consumidor / Productor)
- Recuperación de contraseña
- Control de sesión e inactividad

**Módulo de Productos**

- Creación, edición y eliminación de productos
- Gestión de imágenes mediante Firebase Storage
- Activación e inactivación de productos
- Validación de stock y disponibilidad

**Módulo de Catálogo**

- Visualización dinámica de productos
- Búsqueda por nombre y origen
- Filtros por categoría, precio y región
- Renderización optimizada de información

**Módulo de Compra**

- Carrito de compras persistente por usuario
- Gestión de direcciones de envío
- Selección de empresa de transporte
- Generación de pedidos
- Procesamiento de pagos (simulado)

**Módulo de Productores**

- Gestión de perfil público del productor
- Panel de control (productos, ventas)
- Visualización de pedidos asociados
- Actualización de estado de pedidos

**Módulo de Notificaciones**

- Notificación de cambios de estado de pedidos
- Indicadores visuales en interfaz (frontend)

### 2.3 Tecnologías Utilizadas

El sistema incorpora tecnologías modernas orientadas a desarrollo web escalable:

- **Frontend**: React 18 + Vite
- **Backend**: Python 3.12 + FastAPI + Uvicorn
- **Base de datos**: Firebase Firestore (NoSQL)
- **Autenticación**: Firebase Authentication
- **Almacenamiento**: Firebase Storage
- **Control de Versiones**: GitHub
- **Entorno de desarrollo**: Visual Studio Code

Esta combinación tecnológica permite una arquitectura flexible, con separación de responsabilidades y facilidad de integración entre componentes.

### 2.4 Estructura del sistema

A nivel técnico, el sistema presenta una arquitectura organizada por capas en el backend, siguiendo principios de separación de responsabilidades:

**Backend (FastAPI)**

El backend se estructura en los siguientes componentes:

- Routes: Encargados de recibir las solicitudes HTTP y delegar la lógica al servicio correspondiente.
- Services: Implementan las reglas del negocio, validaciones y flujos operativos del sistema.
- Repositories: Gestionan la interacción con Firebase Firestore, encapsulando la persistencia de datos.
- Models: Definen la estructura de los datos mediante modelos Pydantic.

- Middleware: Incluye validación de autenticación y control de acceso mediante tokens, así como mecanismos de rate limiting.

**Frontend (React)**

El frontend se organiza de forma modular:

- **Contextos globales:**
  - AuthContext (gestión de autenticación)
  - CartContext (estado del carrito)
  - ToastContext (notificaciones UI)
- **Módulos funcionales:**
  - Autenticación
  - Catálogo
  - Carrito
  - Checkout
  - Pedidos
  - Productores
  - Perfil
- **Servicios de integración:**
  - Cliente HTTP (Axios)
  - Configuración Firebase (cliente)

**Servicios externos (Firebase)**

Firebase actúa como componente clave del sistema:

- **Authentication:** Gestión de usuarios y tokens
- **Firestore:** Base de datos NoSQL distribuida
- **Storage:** Almacenamiento de imágenes

**2.5 Flujo de comunicación del sistema**

El flujo general de operación del sistema se basa en la siguiente interacción:

1. El usuario interactúa con el frontend (React)
2. El frontend envía solicitudes HTTP al backend (FastAPI)
3. El backend valida la información y ejecuta la lógica de negocio
4. Se realizan operaciones sobre Firebase (datos o autenticación)
5. El backend retorna la respuesta al frontend
6. El frontend actualiza la interfaz en tiempo real

## 3. Implementación del MVP

La implementación del sistema OrigenX se llevó a cabo bajo un enfoque incremental y evolutivo, partiendo de una versión inicial del MVP con funcionalidades parciales, hacia una solución completamente funcional que cubre el flujo integral del negocio.

A nivel técnico, la implementación se basa en la comunicación entre el frontend (React) y el backend (FastAPI) mediante solicitudes HTTP, donde el cliente consume los servicios expuestos por la API para gestionar usuarios, productos, pedidos y demás funcionalidades del sistema.

### 3.1 Datos de prueba y configuración inicial

Para facilitar la validación del sistema, se configuraron entornos de prueba con datos controlados, permitiendo simular escenarios reales de uso, entre estos se incluyen:

- Usuarios registrados en Firebase Authentication
- Productos almacenados en Firestore con imágenes en Firebase Storage
- Datos de pedidos y carritos generados durante las pruebas

Estos datos permiten validar el comportamiento del sistema en condiciones reales de operación.

### 3.2 Flujo técnico implementado

El sistema implementa múltiples flujos operativos que integran frontend, backend y servicios externos.

**1. Registro de usuario**

- El usuario ingresa sus datos desde el frontend (React)
- El sistema valida los campos en cliente
- Se realiza el registro mediante Firebase Authentication
- Se crea el usuario en la colección correspondiente en Firestore
- Se asigna el rol (Consumidor o Productor) mediante custom claims
- El usuario es autenticado automáticamente y redirigido según su rol

**2. Inicio de sesión**

- El usuario ingresa sus credenciales
- El frontend utiliza Firebase Authentication para validar la información
- Se genera un token de autenticación (JWT)
- El backend valida el token en cada solicitud

- Se controla el acceso según el rol del usuario

**3. Gestión de productos**

- El productor crea productos desde el panel de administración
- El sistema valida la información ingresada
- Las imágenes son almacenadas en Firebase Storage
- Los datos del producto se almacenan en Firestore
- Se permite editar, activar o desactivar productos
- El catálogo se actualiza dinámicamente

**4. Visualización del catálogo**

- El frontend consulta los productos mediante endpoints del backend
- El backend obtiene la información desde Firestore
- Se aplican filtros y criterios de búsqueda
- Se renderiza la información en la interfaz de usuario

**5. Carrito de compras**

- El usuario agrega productos desde el catálogo o detalle
- El sistema valida disponibilidad de stock
- Se almacenan los ítems en la colección del carrito del usuario
- El estado del carrito se mantiene persistente

**6. Proceso de compra (Checkout)**

El flujo de compra se desarrolla en múltiples etapas:

1. Selección de dirección de envío
2. Selección de empresa de transporte
3. Ingreso de datos de pago
4. Confirmación del pedido

El backend valida la información, genera el pedido y descuenta el stock de los productos.

**7. Generación de pedidos**

- Se crea un registro en la colección de pedidos
- Se almacenan los ítems asociados
- Se define el estado inicial del pedido
- Se vacía automáticamente el carrito

**8. Gestión de ventas (Productor)**

- El productor visualiza los pedidos asociados a sus productos
- Puede actualizar el estado del pedido

- Se generan notificaciones al consumidor

### 3.3 Estado de implementación por requerimiento

A continuación, se presenta el estado actual de los requerimientos funcionales del sistema:

| Requerimiento | Estado |
|---|---|
| RF-001 Registro | Implementado |
| RF-002 Login | Implementado |
| RF-003 Roles | Implementado |
| RF-005 Crear producto | Implementado |
| RF-006 Editar producto | Implementado |
| RF-009 Catálogo | Implementado |
| RF-010 Filtros | Implementado |
| RF-011 Detalle producto | Implementado |
| RF-012 Carrito | Implementado |
| RF-013 Pedido | Implementado |
| RF-017 Pago | Implementado (simulado) |

### 3.4 Integración del sistema

La integración entre los diferentes componentes del sistema se realiza de forma coordinada:

- El frontend consume los servicios del backend mediante Axios
- El backend procesa las solicitudes y aplica lógica de negocio
- Firebase gestiona autenticación, persistencia y almacenamiento
- La información fluye en tiempo real entre los componentes
- Esta integración permite garantizar consistencia en los datos, control de acceso seguro y una experiencia de usuario fluida.

## 4. Ejecución de sprints

El desarrollo del sistema OrigenX se llevó a cabo mediante la ejecución iterativa de sprints, siguiendo un enfoque ágil que permitió implementar funcionalidades, identificar fallos y realizar mejoras progresivas hasta alcanzar una versión estable y completamente funcional.
A diferencia de la versión inicial del sistema, donde se evidenciaban limitaciones en módulos clave, la presente iteración refleja la consolidación de los sprints, logrando completar el flujo de negocio y garantizar la estabilidad del sistema mediante procesos de reimplementación y validación continua.

**Sprint 1 – Gestión de usuarios**
**Estado:** Implementado y estabilizado
**Funcionalidades desarrolladas:**

- Registro de usuario
- Inicio de sesión
- Control de acceso por roles

**Evolución:** Durante la implementación inicial se identificaron inconsistencias en la gestión de roles y en la persistencia de sesión. Estas fueron corregidas mediante la integración de Firebase Authentication, permitiendo:

- Validación segura de credenciales
- Gestión de roles mediante custom claims
- Control de sesión con cierre automático por inactividad

**Resultado final:**

- Módulo completamente funcional
- Autenticación estable
- Control de acceso correcto

**Sprint 2 – Gestión de productos**
**Estado:** Implementado y optimizado
**Funcionalidades desarrolladas:**

- Creación de productos
- Edición de productos
- Activación e inactivación
- Gestión de perfil de productor
- Manejo de imágenes

**Evolución:** En la versión inicial, este módulo presentaba limitaciones importantes:

- Ausencia de edición de productos

- Manejo básico de imágenes mediante URL
- Falta de control de estado de productos

A partir de la reimplementación del sistema:

- Se incorporó edición completa de productos
- Se implementó almacenamiento de imágenes mediante Firebase Storage
- Se añadió control de estado (activo/inactivo)
- Se mejoraron validaciones en formularios

**Resultado final:**

- Módulo completamente funcional
- Gestión de productos estable
- Integración correcta con almacenamiento de imágenes

**Sprint 3 – Catálogo y proceso de compra**
**Estado:** Implementado y optimizado
**Funcionalidades desarrolladas:**

- Visualización de catálogo
- Búsqueda y filtros
- Detalle de producto
- Carrito de compras
- Generación de pedidos
- Proceso de pago (simulado)

**Evolución:** Inicialmente, este sprint presentaba múltiples funcionalidades pendientes y fallos en la visualización del catálogo y en el flujo de compra, durante la reimplementación:

- Se corrigieron problemas de renderización
- Se implementaron filtros y búsqueda funcional
- Se desarrolló el carrito persistente
- Se integró el flujo completo de compra
- Se añadió gestión de pedidos y estados

**Resultado final:**

- Flujo end-to-end completamente funcional
- Integración entre módulos correcta
- Experiencia de usuario estable

### 4.1 Consolidación de los sprints

La evolución del sistema permitió pasar de un MVP parcialmente funcional a una solución completa, logrando:

- Integración total entre módulos
- Eliminación de fallos críticos
- Implementación del flujo completo de negocio
- Validación mediante pruebas funcionales y no funcionales

El enfoque ágil adoptado facilitó la identificación temprana de errores, su corrección mediante iteraciones y la mejora progresiva del sistema, culminando en una versión estable y lista para su validación final.

## 5. Pruebas funcionales

### 5.1 Estrategia de pruebas

Las pruebas del sistema OrigenX fueron diseñadas con base en los criterios de aceptación definidos para cada requerimiento funcional, así como en atributos de calidad del software establecidos en el modelo ISO/IEC 25010, en esta iteración se implementó una estrategia de pruebas integral orientada a:

- Validar el funcionamiento completo del sistema
- Garantizar la estabilidad de los módulos
- Verificar la integración entre frontend, backend y servicios externos
- Asegurar la calidad del software en diferentes dimensiones

Se ejecutaron dos tipos principales de pruebas:

**Pruebas funcionales**

Enfocadas en validar el cumplimiento de los requerimientos del sistema:

- Registro y autenticación de usuarios
- Gestión de productos
- Visualización de catálogo
- Carrito de compras
- Generación de pedidos
- Proceso de pago (simulado)
- Gestión de perfil de productor
- Consulta de historial de pedidos

**Pruebas no funcionales**

Orientadas a evaluar atributos de calidad del sistema:

- Compatibilidad: validación en navegadores (Chrome, Firefox, Edge)
- Usabilidad: responsividad en dispositivos (desktop, tablet y móvil)
- Rendimiento: Tiempos de carga y respuesta del sistema
- Seguridad: control de acceso y protección de rutas
- Confiabilidad: manejo de sesiones, errores y consistencia de datos

**5.2 Ejecución de pruebas**

Durante la ejecución de pruebas funcionales se identificaron fallos en algunos escenarios específicos, los cuales fueron documentados y corregidos mediante ajustes en la implementación, posteriormente, se realizó la reejecución de dichas pruebas, obteniendo resultados satisfactorios.

**Métricas de ejecución**

- Total de pruebas ejecutadas: 32
- Pruebas aprobadas (tras reejecución): 32
- Pruebas fallidas en ejecución inicial: 3
- Porcentaje de éxito: 100%

Es importante destacar que este resultado corresponde a la fase final de validación, en la cual todos los escenarios previamente observados fueron corregidos y verificados, evidenciando la estabilidad del sistema en su versión actual.

**Pruebas no funcionales**

- Total de pruebas ejecutadas: 17
- Pruebas aprobadas: 17
- Pruebas fallidas: 0
- Porcentaje de éxito: 100%

Los resultados completos de las pruebas se encuentran documentados en los archivos de evidencia:

- Pruebas funcionales OrigenX (versión final del sistema)
- Pruebas no funcionales del sistema

### 5.3 Evolución de las pruebas

En la versión inicial del sistema, se identificaron múltiples fallos en módulos críticos, particularmente en autenticación, gestión de productos y visualización del catálogo, estos fallos incluyeron:

- Problemas en la gestión de roles
- Limitaciones en la edición de productos
- Fallos en la carga y renderización del catálogo
- Ausencia de funcionalidades completas en el flujo de compra

A través del proceso de reimplementación del sistema, estos problemas fueron corregidos progresivamente, permitiendo:

- Estabilizar los módulos críticos
- Completar las funcionalidades faltantes
- Mejorar la experiencia de usuario
- Garantizar la consistencia de los datos

Como resultado, todas las pruebas inicialmente fallidas fueron corregidas y validadas en fases posteriores, alcanzando un estado final de aprobación total.

### 5.4 Principales hallazgos

Durante la ejecución de las pruebas funcionales del sistema OrigenX, se identificaron una serie de comportamientos relevantes que permitieron mejorar la estabilidad, la experiencia de usuario y la coherencia del sistema. Estos hallazgos corresponden a situaciones reales detectadas durante la validación y posteriormente corregidas mediante iteraciones.

**Gestión de sesión y navegación por roles**
Se identificó un problema crítico en la transición entre roles al cerrar sesión.

- Al cerrar sesión desde el módulo de productor, el sistema mantenía la URL activa (/producer), generando inconsistencias al iniciar sesión con un usuario de rol diferente.
- Esto provocaba mensajes erróneos de “Acceso no autorizado”, aun cuando el usuario había iniciado sesión correctamente.

**Ajuste realizado:**

- Se forzó la redirección a la ruta /login al cerrar sesión.
- Se implementó validación de rutas por rol.
- Se añadió control de carga de rol antes de redireccionar al usuario.

Esto garantizó coherencia en la navegación y control de acceso.

**Restricción en selección de regiones (perfil de productor)**

Durante la edición del perfil comercial, se evidenció una limitación en la selección de regiones.

- El sistema solo permitía seleccionar un conjunto limitado de regiones (Huila, Antioquia y Nariño).
- Esto restringía la representatividad de productores de otras zonas cafeteras.

**Ajuste realizado:**

- Se reemplazó el campo “región” por “departamento”.
- Se ampliaron las opciones incluyendo todos los departamentos cafeteros de Colombia.
- Se unificó este cambio en perfil de productor y catálogo.

Esto mejoró la consistencia de datos y cobertura del sistema.

**Carga y visualización de imágenes en productos**

Se detectó un comportamiento inconsistente en la carga de imágenes al crear productos.

- Al crear un producto por primera vez, la imagen no se visualizaba correctamente.
- Sin embargo, al editar el producto o agregar imágenes adicionales, el sistema funcionaba correctamente.

**Ajuste realizado:**

- Se corrigió la lógica de renderización inicial de imágenes.
- Se habilitó correctamente la opción de carga en el primer registro.
- Se validó la persistencia de imágenes en Firebase Storage.

Esto aseguró la correcta visualización en el catálogo desde la creación inicial.

**Validaciones en formularios**

Se evidenció la necesidad de fortalecer las validaciones en formularios de registro y productos.

- Casos detectados:
    - Campos obligatorios no controlados correctamente
    - Validaciones incompletas en formato de datos
    - Retroalimentación insuficiente al usuario

**Ajuste realizado:**

- Se implementaron validaciones en frontend y backend.
- Se añadieron mensajes claros de error.
- Se bloquearon envíos de formularios con datos inválidos.

Esto mejoró significativamente la experiencia de usuario y la integridad de los datos.

**Integración de funcionalidades en flujo completo**
Durante la validación del sistema se evidenció que múltiples módulos dependían de la correcta sincronización entre frontend, backend y Firebase.

- Se identificaron riesgos en:
  - Persistencia de datos en tiempo real
  - Consumo de API
  - consistencia en estados del sistema

**Ajuste realizado:**

- Se optimizó la comunicación entre servicios.
- Se validaron respuestas del backend antes de renderizar.
- Se controlaron estados globales mediante contextos en frontend.

Esto permitió garantizar la estabilidad del flujo end-to-end.

### 5.5 Conclusión de pruebas

Los hallazgos identificados durante las pruebas no solo permitieron corregir errores puntuales, sino también mejorar aspectos estructurales del sistema, particularmente en la gestión de sesiones, validación de datos, manejo de imágenes e integración de componentes, como resultado de estos ajustes, el sistema alcanzó un estado estable, logrando la aprobación total de los escenarios de prueba definidos.

## 6. Despliegue del sistema

### 6.1 Entorno de ejecución

El sistema OrigenX se despliega en un entorno de prueba funcional que permite validar la interacción completa entre los componentes del sistema: frontend, backend y servicios externos, a diferencia de la versión inicial, donde el sistema operaba con una configuración local básica y base de datos interna, la versión actual integra servicios en la nube mediante Firebase, permitiendo una arquitectura más escalable y cercana a un entorno productivo.

### 6.2 Configuración del backend

El backend del sistema está desarrollado en FastAPI y se ejecuta en un servidor local mediante Uvicorn.

**Ejecución del servidor**
uvicorn main:app --reload

**Dirección de acceso**
- API Backend: http://localhost:8000
- Documentación automática (Swagger): http://localhost:8000/docs

El backend se encarga de:
- Procesar solicitudes HTTP del frontend
- Ejecutar la lógica de negocio
- Validar autenticación mediante tokens
- Gestionar operaciones con Firebase

**6.3 Configuración del frontend**
El frontend es una aplicación SPA desarrollada con **React + Vite**, ejecutada en entorno local durante la fase de pruebas.

**Ejecución del cliente**
npm install
npm run dev

**Dirección de acceso**
- Aplicación web: http://localhost:5173

El frontend consume los servicios del backend mediante llamadas HTTP (Axios), permitiendo la interacción dinámica con el sistema.

**6.4 Integración con Firebase**
El sistema utiliza Firebase como plataforma de servicios externos para la gestión de autenticación, datos y almacenamiento.

**Servicios utilizados:**
- **Firebase Authentication:** Gestión de usuarios, autenticación y tokens JWT
- Firestore: Base de datos NoSQL utilizada para almacenar información de usuarios, productos, pedidos y carritos
- Firebase Storage: Almacenamiento de imágenes asociadas a productos

La integración se realiza mediante SDK oficial de Firebase, tanto en frontend como backend, garantizando comunicación segura y consistente.

### 6.5 Exposición del sistema en entorno de prueba

Para permitir pruebas externas y validación en distintos entornos, el backend puede exponerse mediante herramientas como **ngrok**, generando una URL pública temporal.

**Ejemplo de exposición:**
ngrok http 8000

Esto permite:

- Acceso remoto a la API
- Pruebas en dispositivos externos
- Validación de integración en condiciones reales

### 6.6 Flujo de despliegue

El despliegue del sistema sigue el siguiente flujo:

1. Inicialización del backend (FastAPI)
2. Configuración de conexión con Firebase
3. Ejecución del frontend (React)
4. Consumo de API desde el cliente
5. Validación de funcionalidades en entorno local o expuesto

### 6.7 Consideraciones técnicas

Durante el despliegue se tuvieron en cuenta los siguientes aspectos:

- Manejo de variables de entorno para credenciales de Firebase
- Configuración de CORS en el backend para permitir consumo desde el frontend
- Validación de tokens en cada solicitud protegida
- Control de errores en comunicación entre frontend y backend

### 6.8 Estado del despliegue

El sistema se encuentra desplegado en un entorno de prueba funcional, permitiendo:

- Ejecución completa del flujo de negocio
- Validación de todos los módulos implementados
- Integración efectiva entre componentes

No se realizó un despliegue en infraestructura productiva, dado que el objetivo del proyecto corresponde al desarrollo y validación de un MVP funcional.

## 7. Gestión del proyecto

La gestión del proyecto se llevó a cabo mediante el uso de GitHub Projects, herramienta que permitió organizar, dar seguimiento y controlar el avance de las tareas asociadas a cada uno de los sprints definidos, a través de esta plataforma se gestionaron los requerimientos funcionales del sistema, estableciendo un flujo de trabajo basado en estados que facilitaron la visualización del progreso del proyecto.

### 7.1 Organización del tablero

El tablero de GitHub Projects se estructuró en las siguientes columnas:

- Backlog: requerimientos pendientes de desarrollo
- In Progress: tareas en ejecución
- Done: funcionalidades implementadas y validadas

Cada requerimiento fue representado como una tarea dentro del tablero, permitiendo:

- Asignación de responsables
- Seguimiento del estado de desarrollo
- Relación con los sprints definidos
- Control del avance del proyecto

### 7.2 Trazabilidad de requerimientos

El uso de GitHub Projects permitió mantener trazabilidad entre:

- Requerimientos funcionales (RF)
- Historias de usuario (HU)
- Tareas de desarrollo
- Resultados de pruebas

Esta trazabilidad facilitó la identificación de:

- Funcionalidades implementadas
- Elementos pendientes
- Requerimientos que requirieron ajustes durante el desarrollo

### 7.3 Evidencia de gestión

Como evidencia del proceso de gestión del proyecto, se incluyen capturas del tablero de GitHub Projects en diferentes momentos del desarrollo, donde se observa:

- La evolución de los requerimientos a lo largo de los sprints
- El cambio de estado de las tareas (Backlog → In Progress → Done)

- La consolidación final de funcionalidades implementadas

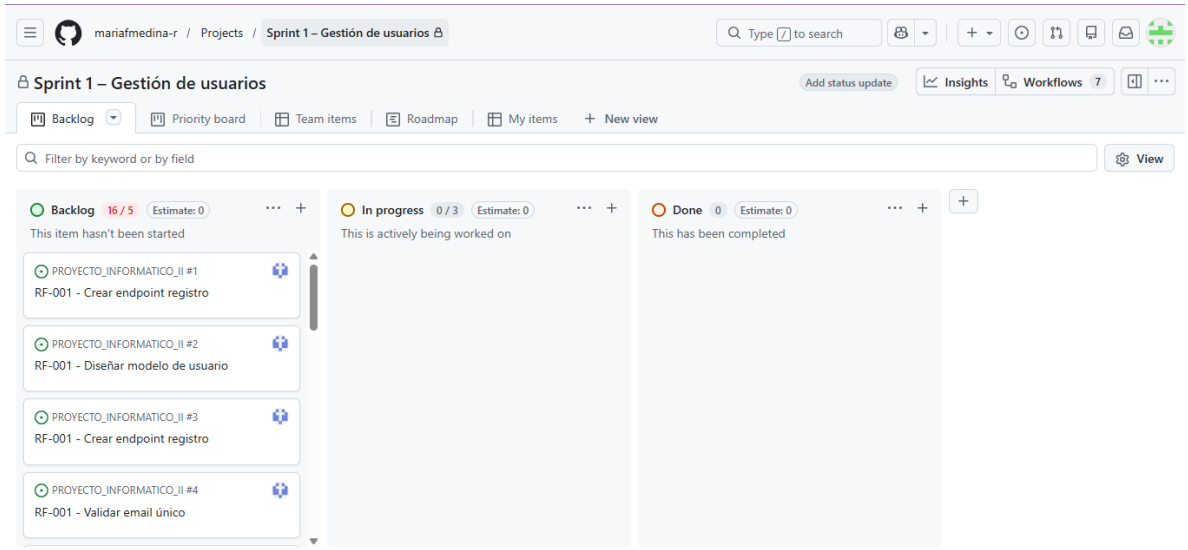


*Ilustración 1- Tablero de gestión de tareas – Sprint 1 (GitHub Projects)*

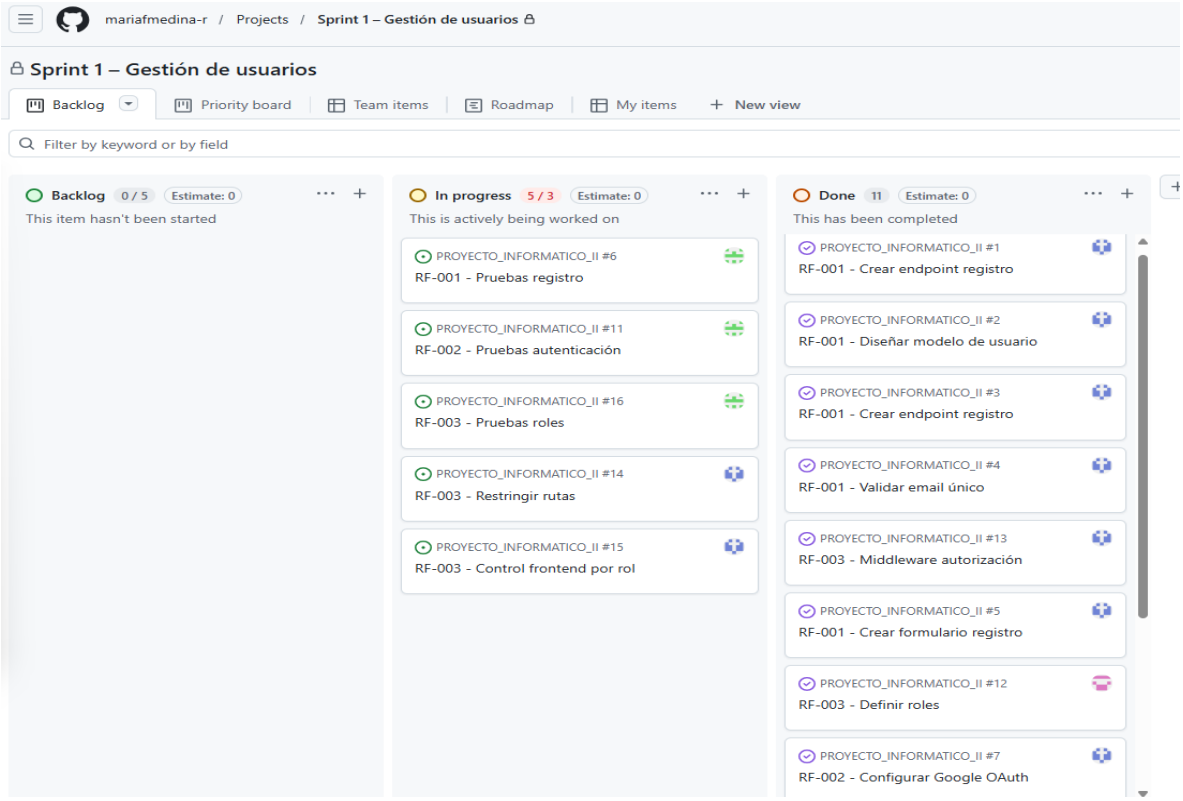


*Ilustración 2 - Tablero de gestión de tareas – Sprint 1 Avances*

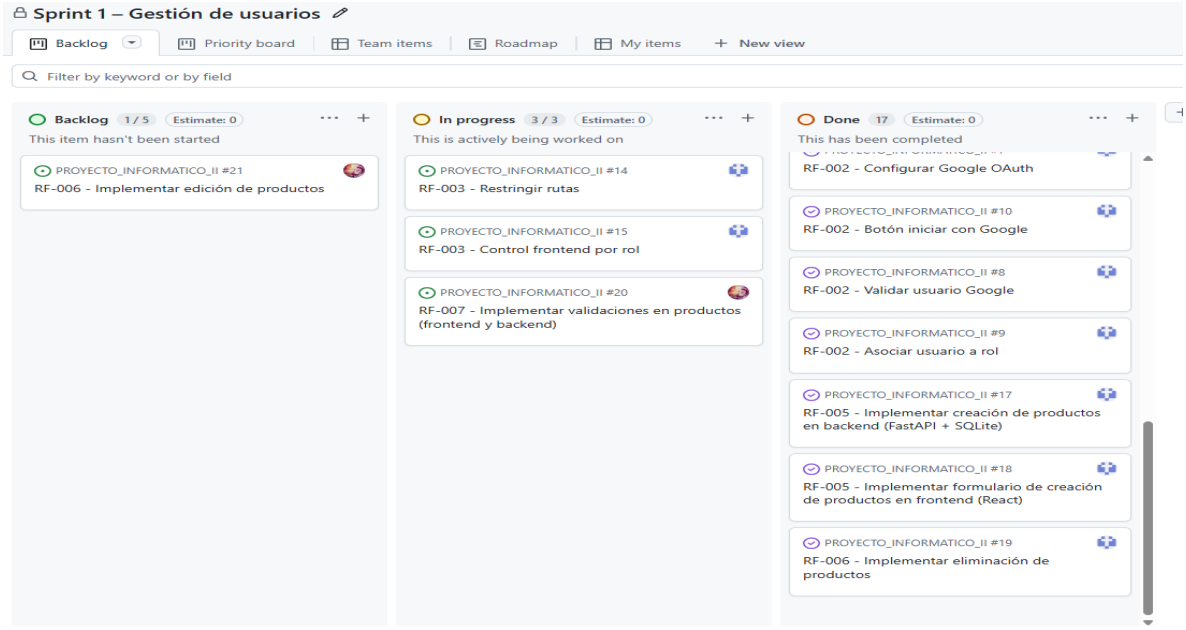


*Ilustración 3 - Estado inicial del tablero de GitHub Projects con avance del Sprint 1 y tareas en desarrollo del Sprint 2*

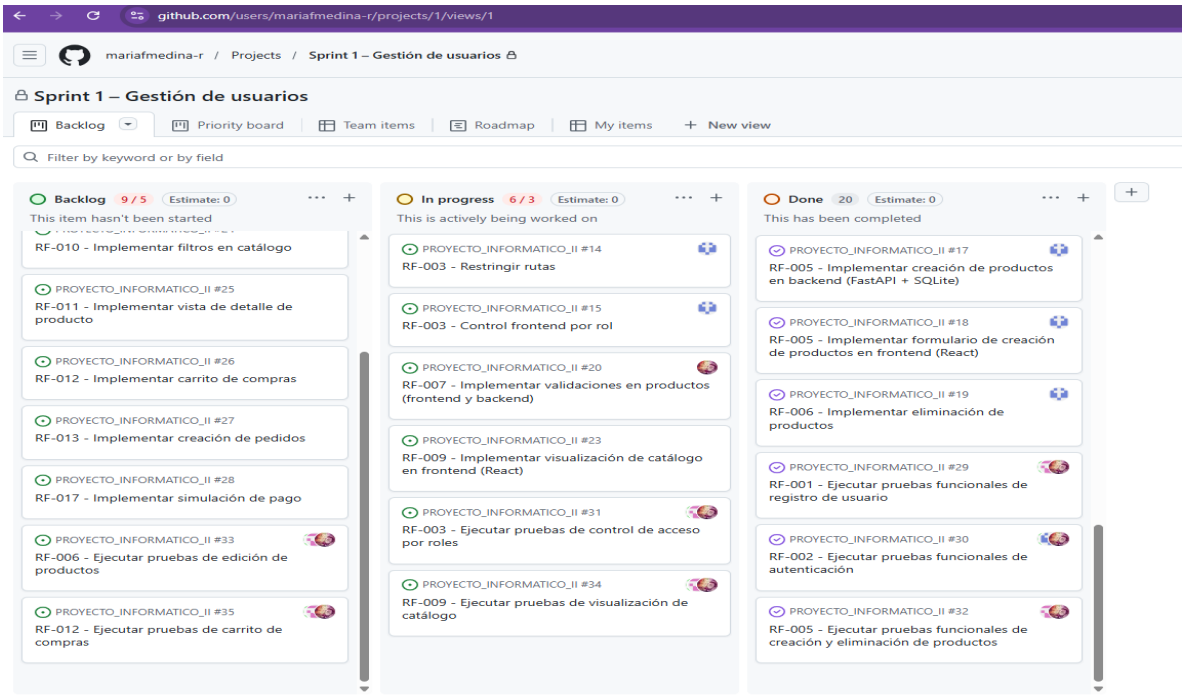


*Ilustración 4 - Tablero de gestión del proyecto en GitHub Projects con evidencia de ejecución de los Sprints 1, 2 y 3*

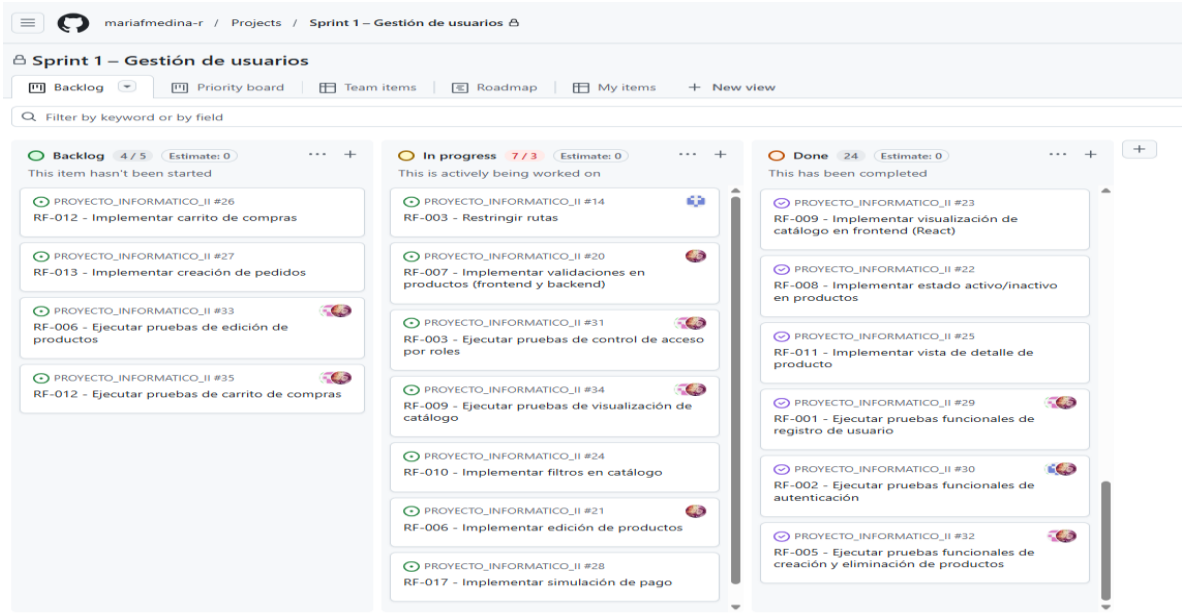


*Ilustración 5 - Estado actual del tablero de GitHub Projects con la ejecución de los sprints, evidenciando requerimientos implementados, en progreso y pendientes*

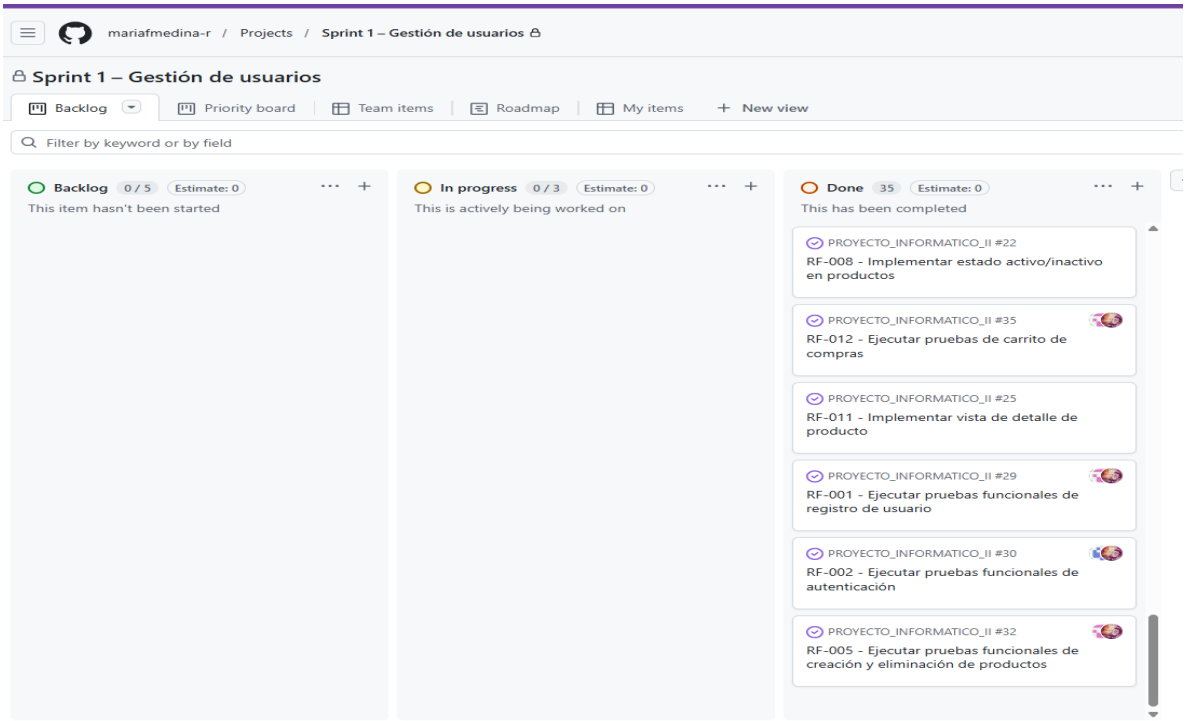


*Ilustración 6 - Estado final del tablero de GitHub Projects evidenciando la finalización de las tareas del Sprint 1 y la ejecución completa de los requerimientos funcionales*

### 7.4 Consideraciones

GitHub Projects fue utilizado como alternativa a herramientas como JIRA, permitiendo cumplir con los objetivos de gestión de tareas, seguimiento y control del proyecto, manteniendo un enfoque ágil y colaborativo.

## 8. Roles y responsabilidades del equipo

Durante el desarrollo del sistema OrigenX**,** el equipo de trabajo asumió diferentes roles con el fin de garantizar la correcta ejecución de las actividades de análisis, desarrollo, pruebas y documentación, a lo largo del proyecto se presentó una redistribución de responsabilidades, particularmente en el rol de desarrollo, lo que permitió la reimplementación completa del sistema y la consolidación de la solución final.

### 8.1 Asignación de roles

| Integrante | Rol | Responsabilidad |
|---|---|---|
| **Edward Rubio Rodríguez** | Developer / DevOps | Desarrollo completo del sistema, implementación del backend y frontend, integración con Firebase, despliegue en entorno de prueba e implementación de arquitectura. |
| **Francisco Stiven Tabares Ussa** | Developer (fase inicial) | Participación en la fase inicial hasta Sprint 2 del desarrollo del sistema |
| **Leonardo Martínez Franco** | Product Owner | Definición y priorización de requerimientos, gestión del backlog y validación funcional |
| **María Fernanda Medina Ramírez** | QA Engineer / Documentación | Diseño y ejecución de pruebas, validación del sistema y elaboración de documentación técnica |

### 8.2 Consideraciones del equipo

El cambio en la asignación del rol de desarrollo representó un punto clave en la evolución del proyecto, ya que permitió:

- Replantear la arquitectura del sistema
- Corregir limitaciones de la implementación inicial
- Completar funcionalidades críticas
- Mejorar la calidad del software

Este ajuste evidenció la capacidad del equipo para adaptarse a cambios y tomar decisiones técnicas orientadas a la mejora continua del sistema.

## 9. Conclusión

El desarrollo del sistema OrigenX permitió evidenciar un proceso completo de evolución técnica, pasando de una implementación inicial con funcionalidades parciales a una solución integral, estable y validada, durante las primeras fases del proyecto se identificaron limitaciones importantes relacionadas con la arquitectura, la gestión de datos y la integración de módulos, lo que impactaba directamente la funcionalidad del sistema, sin embargo, mediante la reimplementación del aplicativo y la adopción de tecnologías más robustas como Firebase, se logró construir una arquitectura desacoplada, escalable y alineada con buenas prácticas de desarrollo.

El enfoque iterativo basado en sprints permitió no solo desarrollar funcionalidades, sino también identificar fallos reales durante las pruebas funcionales y no funcionales, los cuales fueron documentados, corregidos y validados en reejecuciones posteriores, este proceso garantizó la mejora progresiva del sistema y la consolidación de cada uno de sus módulos.

Como resultado, el sistema actual cubre de manera completa el flujo del negocio, incluyendo autenticación de usuarios, gestión de productos, visualización de catálogo, carrito de compras, generación de pedidos y procesamiento de pagos simulados, evidenciando una integración efectiva entre frontend, backend y servicios externos.

En conclusión, el proyecto no solo cumple con los objetivos planteados para el desarrollo del MVP, sino que demuestra un proceso de aprendizaje, adaptación y mejora continua, reflejando un enfoque de ingeniería orientado a la calidad del software y la entrega de valor.

## 10. Repositorio

El código fuente del sistema **OrigenX,** junto con la documentación técnica y evidencias del proceso de desarrollo, se encuentra disponible en el siguiente repositorio de GitHub:

https://github.com/mariafmedina-r/PROYECTO_INFORMATICO_II

En este repositorio es posible consultar:

- La estructura completa del sistema (frontend y backend)
- La organización modular del código (routes, services, repositories)
- Los archivos de configuración y dependencias del proyecto
- La documentación técnica y guías de ejecución
- El historial de cambios y evolución del desarrollo

### 10.1 Entorno de ejecución

La ejecución del sistema se realizó en las siguientes direcciones:

- Backend (API): http://127.0.0.1:8000
- Documentación API: http://127.0.0.1:8000/docs
- Frontend**:** http://localhost:5173

### 10.2 Consideraciones

El repositorio permite la revisión completa del proyecto, facilitando su validación, replicación y posible extensión, cumpliendo con los requerimientos establecidos para la entrega del avance.

## 11. Bibliografía


- Pressman, R. S., & Maxim, B. R. (2020). Ingeniería de software: Un enfoque práctico (9ª ed.). McGraw-Hill.
- Sommerville, I. (2011). Ingeniería de software (9ª ed.). Pearson.
- Universidad Autónoma de Occidente. (2026). Guía Entrega - Justificación de soluciones.
- ISO/IEC. (2011). ISO/IEC 25010: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE).
- FastAPI. (2024). *FastAPI Documentation*. https://fastapi.tiangolo.com
- React. (2024). *React Documentation*. https://react.dev
- Google. (2024). *Firebase Documentation*. https://firebase.google.com/docs